**GIT & GITHUB PRACTICAL ASSIGNMENT**

**Modules I & II — Version Control and Collaboration**

**Name:** Sufiya Sultana
**Project:** Personal Portfolio Website

**1. Introduction**

The objective of this assignment was to understand and practically implement Git and GitHub concepts such as version control, repository creation, branching, committing, merging, conflict resolution, and collaboration.

For Part A, I created a simple personal portfolio website using HTML and CSS. I used Visual Studio Code for development and GitHub to manage my repository.

**2. Repository and Branch Structure**

I initialized a Git repository and created README.md and .gitignore files. After making my initial commit, I connected the local repository to GitHub.

I created three feature branches:

| Branch | Description |
|---|---|
| feature/homepage | Created the homepage and About Me section. |
| feature/styling | Added CSS styling and improved page spacing. |
| feature/navbar | Created navigation links and a footer. |

Each feature branch contained at least two commits.

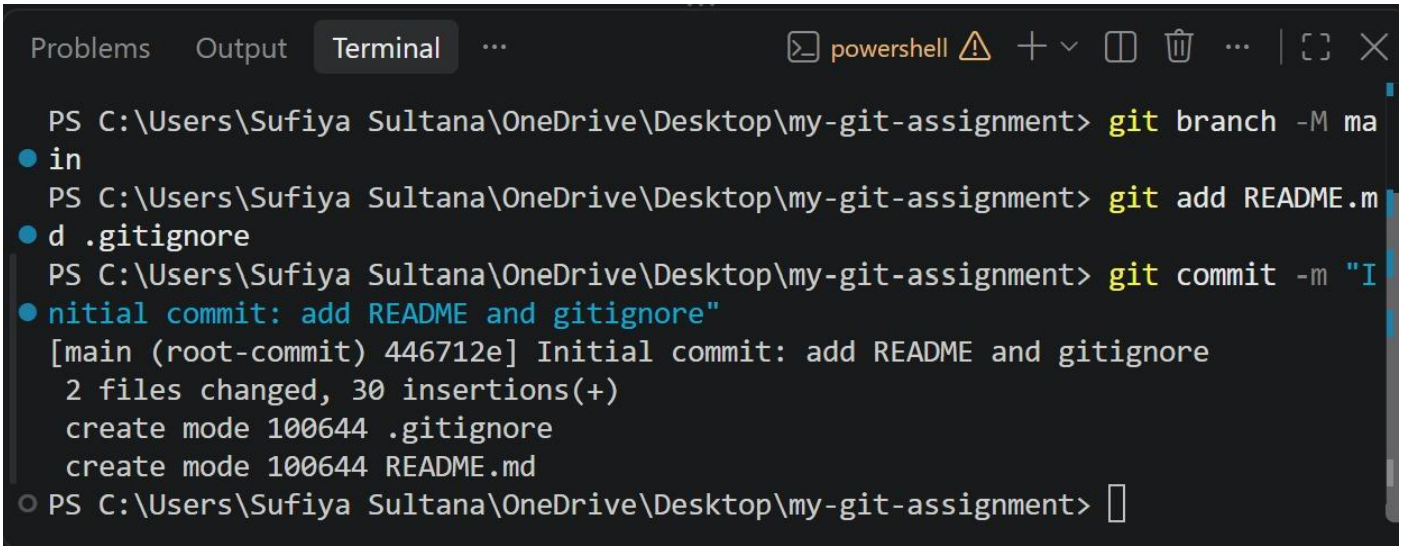
**GitHub Repository:**
my-git-assignment

**3. Branch Merging**

I performed two types of merges during the assignment.

**Fast-forward merge:** I merged the feature/homepage branch into main. Since the main branch had not changed after creating the feature branch, Git moved the branch pointer forward without creating a separate merge commit.

**Three-way merge:** I merged the feature/styling branch using the --no-ff option. Since the branches had different commit histories, Git combined their changes and created a separate merge commit.



```
PS C:\Users\Sufiya Sultana\OneDrive\Desktop\my-git-assignment> git log --onelin
e --graph --all
* 5f357de (HEAD -> main, origin/main) Remove remaining conflict markers
*   799af88 Merge navbar branch and resolve README conflict
|\
| * e376810 (origin/feature/navbar, feature/navbar) Update project heading on n
avbar branch
| * b9df8f4 Add footer to navigation component
| * 346759a Add navigation links
* | f556006 Update project heading on main
* |   df4fc70 Merge styling branch
:...skipping...
* 5f357de (HEAD -> main, origin/main) Remove remaining conflict markers
*   799af88 Merge navbar branch and resolve README conflict
|\
| * e376810 (origin/feature/navbar, feature/navbar) Update project heading on n
avbar branch
| * b9df8f4 Add footer to navigation component
| * 346759a Add navigation links
* | f556006 Update project heading on main
* |   df4fc70 Merge styling branch
|\ \
| * | 6ca2474 (origin/feature/styling, feature/styling) Improve page spacing an
d headings
| * | 6b44295 Add basic website styling
| |/
* | 94abdcc Update README with project details
* | 3d81945 (origin/feature/homepage, feature/homepage) Add about me section
* | fe9623d Add homepage structure
|/
* 446712e Initial commit: add README and gitignore
~
~
```

**4. Merge Conflict and Resolution**

I deliberately created a merge conflict by modifying the same heading in README.md on two different branches.

On the main branch, I changed the heading to "My Personal Portfolio Project", while on the navbar branch, I changed it to "My GitHub Portfolio Assignment".

When I attempted to merge the branches, Git reported a conflict because both branches contained different modifications to the same line.

I resolved the conflict by choosing "My Personal Portfolio Project" as the final heading and removing the conflict markers.

After resolving the conflict, I staged the README file and committed the changes.

README.md > abc # My GitHub Portfolio Assignment > abc ## Author

Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Char

```
<<<<<<< HEAD (Current Change)
# My Personal Portfolio Project
=======
# My GitHub Portfolio Assignment
>>>>>>> feature/navbar (Incoming Change)

This project is a simple personal portfolio website.

```

Problems Output Terminal ··· powershell

```
PS C:\Users\Sufiya Sultana\OneDrive\Desktop\my-git-assignment> git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 7 commits.
  (use "git push" to publish your local commits)
PS C:\Users\Sufiya Sultana\OneDrive\Desktop\my-git-assignment> git merge featur
e/navbar
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Sufiya Sultana\OneDrive\Desktop\my-git-assignment>
```

```
PS C:\Users\Sufiya Sultana\OneDrive\Desktop\my-git-assignment> git add README.m
d
PS C:\Users\Sufiya Sultana\OneDrive\Desktop\my-git-assignment> git commit -m "U
pdate README with project details"
[main 94abdcc] Update README with project details
 1 file changed, 2 insertions(+), 1 deletion(-)
PS C:\Users\Sufiya Sultana\OneDrive\Desktop\my-git-assignment> git merge --no-f
f feature/styling -m "Merge styling branch"
Merge made by the 'ort' strategy.
 style.css | 20 ++++++++++++++++++++
 1 file changed, 20 insertions(+)
 create mode 100644 style.css
```

**5. Part B — Cloning and Repository Exploration**

For Part B, I cloned the existing GitHub Hello-World repository.

I used the following commands to explore its history:

| Git Command | Purpose |
|---|---|
| git log --oneline --graph --all | Displayed commit history and branch structure. |
| git branch -a | Displayed local and remote branches. |
| git log -p -3 | Displayed detailed changes from the latest three commits. |

| Git Command | Purpose |
| --- | --- |
| git blame README | Identified the commit and author responsible for the README line. |

This activity helped me understand how Git records changes made by developers over time.

**Cloned Repository:**
Hello-World Repository

**6. Reflection Questions**

**Q1. What is the difference between a fast-forward merge and a three-way merge?**

A fast-forward merge simply moves the main branch pointer forward when there are no separate commits on main. A three-way merge combines changes from diverged branches and creates a merge commit. In my project, the homepage branch used a fast-forward merge, while the styling branch used a three-way merge.

**Q2. How did you resolve the merge conflict?**

The conflict occurred because I changed the same README heading differently on two branches. Git could not automatically decide which version to keep. I manually selected the preferred heading, removed the conflict markers, saved the file, and committed the resolution.

**Q3. How was the cloned repository different from your own?**

My repository had three feature branches with commits organized according to specific development tasks. The cloned Hello-World repository had an existing history that I had not created myself. Exploring it helped me understand how to examine another project's commits and identify previous contributors.

**Q4. What problems can force-pushing create?**

Force-pushing rewritten history can overwrite commits on a remote branch. If another developer has already downloaded the earlier commits, their local history may no longer match the remote repository. This can create conflicts and make collaboration difficult.

**7. Conclusion**

This assignment provided practical experience in using Git and GitHub for version control. I learned how to create repositories, manage feature branches, perform different types of merges, resolve conflicts, and explore the history of an existing project. These skills are important for maintaining organized projects and collaborating with other developers.